

CSE420 — Compiler Design

Chapter 3: Lexical Analysis

Prepared by Azmari Sultana

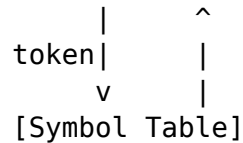
Contents

3.1 The Role of the Lexical Analyzer	2
3.1.1 Lexical Analysis versus Parsing	3
3.1.2 Tokens, Patterns, and Lexemes	3
3.1.3 Attributes for Tokens	5
3.1.4 Lexical Errors	6
Lexical Analyzer Generation	6
3.1.5 Practice Exercises (Section 3.1)	7
3.2 Input Buffering	7
3.2.1 Buffer Pairs	8
3.2.2 Sentinels	9
3.3 Specification of Tokens	10
3.3.1 Strings and Languages	10
3.3.2 Operations on Languages	11
3.3.3 Regular Expressions	11
3.3.4 Regular Definitions	13
3.3.5 Extensions of Regular Expressions	14
3.3.6 Practice Exercises (Section 3.3)	15
3.4 Recognition of Tokens (Introduction)	16

3.1 The Role of the Lexical Analyzer

The lexical analyzer (also called **scanner** or **lexer**) is the **first phase** of a compiler. It reads the source program character by character, groups characters into **lexemes**, converts each lexeme into a **token**, and passes tokens to the parser.

```
source program --> [Lexical Analyzer] <--getNextToken--> [Parser] --
> semantic analysis
```



- The parser calls the lexer using `getNextToken()`.
- The lexer reads characters, finds the next lexeme, builds a token, and returns it.
- When the lexer finds an identifier, it inserts/looks it up in the **symbol table**.

Additional jobs of a lexical analyzer

- Removes comments and whitespace.
- Keeps track of line numbers (for error messages).
- May expand macros (if a macro-preprocessor is used).
- Detects simple, obvious lexical errors.

Scanning vs. Lexical Analysis Proper

Scanning	Lexical Analysis Proper
Removes comments, whitespace	Identifies lexemes, builds tokens
Does not produce tokens	Produces tokens
Simple/cheap	More complex

Exam Q&A

Q1. What is a lexical analyzer? A: The first phase of a compiler; reads source code, identifies lexemes, converts them into tokens, and sends tokens to the parser.

Q2. Why is a lexical analyzer needed? A: Converts code into tokens; removes comments/whitespace; detects simple lexical errors; simplifies parsing; improves compiler efficiency.

Q3. List the steps performed by a lexical analyzer. A: (1) Read source code → (2) Identify next lexeme → (3) Match against token pattern → (4) Generate token → (5) Update symbol table if identifier → (6) Return token to parser → (7) Repeat till end of file.

Q4. How do the lexer and parser interact? A: The parser calls `getNextToken()`; the lexer scans ahead, finds the next lexeme, converts it to a token, and returns it to the parser.

Q5. How does the lexer interact with the symbol table? A: For every identifier lexeme, it checks whether it is already in the symbol table; if not, inserts it; then returns a **pointer to the symbol-table entry** as the token's attribute.

3.1.1 Lexical Analysis versus Parsing

Lexical analysis and parsing are kept as **separate phases** for three reasons — remember **S-E-P**:

1. **Simplicity** — the parser does not need to deal with whitespace/comments.
2. **Efficiency** — specialized buffering/scanning techniques speed up compilation.
3. **Portability** — device-specific issues stay isolated inside the lexer.

Lexical Analysis	Parsing
Converts characters into tokens	Checks whether tokens follow the grammar
Happens first	Happens after lexical analysis

Exam Q&A

Q6. Why are lexical analysis and parsing separated into two phases? A: For simplicity, efficiency, and portability (**S-E-P**) — see above.

3.1.2 Tokens, Patterns, and Lexemes

Term	Meaning
Token	A pair (token name, optional attribute). An abstract symbol used by the parser.
Pattern	The rule describing what strings can form a lexeme of a token (often a regular expression).
Lexeme	The actual sequence of characters in the source code matching a pattern.

Example: for id, pattern = letter (letter|digit)*, lexeme = total.

Common token classes: keywords, operators, identifiers (all under one id token), constants (numbers/strings), punctuation symbols.

Reference table — typical tokens, patterns, and sample lexemes

Token	Informal Pattern	Sample Lexemes
if	characters i, f	if
else	characters e, l, s, e	else
comparison	< or > or <= or >= or == or !=	<=, !=
id	letter followed by letters and digits	pi, score, D2
number	any numeric constant	3.14159, 0, 6.02e23

Token	Informal Pattern	Sample Lexemes
literal	anything but ", surrounded by "s	"core dumped"

Simulation – Tokenizing a statement

Input: `sum = a + b1;`

Step	lexemeBegin...forward scans	Lexeme found	Token generated
1	s-u-m (stops at space)	sum	<ID, ptr to sum>
2	skip space, read =	=	<ASSIGN_OP>
3	skip space, read a (stops at space)	a	<ID, ptr to a>
4	skip space, read +	+	<PLUS_OP>
5	skip space, b-1 (stops at ;)	b1	<ID, ptr to b1>
6	read ;	;	<SEMICOLON>

Output token stream: <ID, sum> <ASSIGN_OP> <ID, a> <PLUS_OP> <ID, b1> <SEMICOLON>

Exam Q&A

Q7. Differentiate Token, Pattern, and Lexeme. A: Token = category (e.g. ID); Pattern = rule for that category (e.g. letter(letter|digit)*); Lexeme = actual text matched (e.g. total).

Q8. Tokenize `printf("Total=%d\n", score);` A:

Lexeme	Token
printf	ID
(	LPAREN
"Total=%d\n"	LITERAL
,	COMMA
score	ID
)	RPAREN
;	SEMICOLON

Q9. What are the common classes of tokens in a language? A: Keywords, operators, identifiers, constants (numbers/literals), punctuation symbols.

Q10. What is the final output of a lexical analyzer? A: A stream of tokens sent one by one to the parser, e.g. <ID><ASSIGN><NUMBER><SEMICOLON>.

Note — Recognizing a token is not always immediate

In fixed-format Fortran, blanks are ignored, so:

```
D0 5 I = 1.25
```

looks at first like a D0 keyword — but it is actually the identifier D05I being assigned 1.25 (a do-statement needs a **comma**, not a dot, e.g. D0 5 I = 1,25). The lexer cannot decide the first lexeme is D05I until it sees the **dot** after the 1. This shows lexeme recognition sometimes needs to look arbitrarily far ahead.

3.1.3 Attributes for Tokens

A token may carry an **attribute value** describing the specific lexeme matched (since one pattern can match many lexemes, e.g. number matches 0 and 100).

- **Token name** → used by the parser to check grammar.
- **Attribute value** → used later, in translation/code generation.
- A token is assumed to have **at most one** attribute (though it may be a structured value).
- For id, the attribute is a **pointer to the symbol-table entry** (which stores name, type, first-occurrence location, etc.).
- Keywords, operators, and punctuation usually need **no** attribute.

Example — Fortran statement `E = M * C ** 2`

Token	Attribute
id	pointer to entry for E
assign_op	—
id	pointer to entry for M
mult_op	—
id	pointer to entry for C
exp_op	—
number	integer value 2

Exam Q&A

Q11. Why are attributes needed for tokens? A: Because one pattern can match many lexemes; the token name alone cannot tell later compiler phases which exact lexeme was found, so an attribute value is attached.

Q12. What does the lexer return to the parser? A: A token name, and (if needed) an attribute value.

Q13. What is stored as the attribute of an id token, and why? A: A pointer to its symbol-table entry — this avoids storing all identifier details (name, type, location) repeatedly and saves memory.

Q14. “Token name influences parsing, attribute value influences translation” — explain. A: The token name tells the parser whether the grammar is being followed; the attribute value is used later (semantic analysis, code generation) to work with the real data.

3.1.4 Lexical Errors

A **lexical error** occurs when the lexer cannot match the remaining input to any token pattern.

- Some wrong inputs are still *valid lexemes* (e.g. `fi` is a valid `id`, even if the programmer meant `if`) — the lexer cannot detect this; a later phase (parser) must.
- If truly no pattern matches, the lexer uses **panic mode recovery**: delete characters one by one until a valid token is found.

Other error-recovery actions — Mnemonic: P-DIRT

Technique	Meaning	Example
Panic mode	Delete characters until a valid token is found	—
Delete	Remove one incorrect character	<code>ab\$c</code> → <code>abc</code>
Insert	Add a missing character	<code>whle</code> → <code>while</code>
Replace	Replace a wrong character	<code>lf</code> → <code>if</code>
Transpose	Swap two adjacent characters	<code>fi</code> → <code>if</code>

Finding the *minimum* number of corrections is not used in practice — too expensive computationally.

Exam Q&A

Q15. What is a lexical error? A: An error that occurs when the lexer cannot recognize a valid token from the remaining input.

Q16. Why can't a lexer always detect an error, using the `fi` example? A: `fi` (`a == f(x)`) — `fi` matches the `id` pattern, so the lexer must return `id`; it cannot know the programmer meant `if`. The error is caught later (usually by the parser).

Q17. What is panic mode recovery? A: The simplest lexical-error recovery technique — delete characters one at a time from the input until a well-formed token is found.

Q18. List lexical error-recovery techniques. A: Panic mode, delete a character, insert a character, replace a character, transpose two characters (**P-DIRT**).

Q19. Why is minimum-edit correction not used in practice? A: It is accurate but computationally too expensive; simpler techniques like panic mode are preferred.

Lexical Analyzer Generation

Writing a lexical analyzer by hand means building the automaton for every token's regular expression, plus code for file reading and input buffering — tedious, and the implementation process is basically the same regardless of the actual patterns used. So in practice, nobody writes lexers by hand.

Instead, a **lexical-analyzer generator tool** (e.g. **LEX** or **FLEX**) is used: - The programmer writes a regular expression for each token. - For each expression, a short action (usually C code) says what token to return when that pattern matches. - The generator converts all this into a working lexer: a **transition table** plus an **automaton simulator** that uses `lexemeBegin/forward` pointers over the input buffer.

Lex program --> [Lex compiler] --> Transition table + Actions --
 > [Automaton simulator] <--> Input buffer

Important rule: keyword patterns (e.g. if, while) usually also match the identifier pattern (since keywords are just letters). To stop keywords being wrongly recognized as identifiers, the generator uses the rule: *the first pattern in the list that matches is used*. So **keyword patterns must be listed before the identifier pattern** in the specification (and the same idea applies to any pattern whose matches are a subset of another's).

Exam Q&A

Q20. Why are lexical analyzers usually built using a generator tool instead of by hand? A: Building the automaton and buffering code by hand for every token pattern is repetitive and tedious; a generator (like Lex/Flex) automates this — the programmer only supplies regular expressions and short actions.

Q21. Why must keyword patterns be listed before the identifier pattern in a Lex specification? A: A lexer generator picks the *first* pattern in the list that matches. Since a keyword also matches the identifier pattern, keywords need higher priority, or they would be wrongly tokenized as identifiers.

3.1.5 Practice Exercises (Section 3.1)

- Divide the C program

```
float limitedSquare(x) float x {
    /* returns x-squared, but never more than 100 */
    return (x<=-10.0 || x>=10.0)?100:x*x;
}
```

into its lexemes, and decide which lexemes should carry an attribute value (and what that value should be).

- Tagged languages like HTML/XML differ from ordinary programming languages because punctuation (tags) is either very numerous (HTML) or user-definable (XML), and tags can carry parameters. For a short HTML snippet containing text, a tag, an tag, and a link , decide how it should be divided into lexemes, and which lexemes need attribute values.

3.2 Input Buffering

Reading the source program **character-by-character** from disk is slow. Also, the lexer often needs **lookahead** — e.g., it cannot be sure an identifier has ended until it sees a non-letter/non-digit character; in C, < might be the start of <=. To handle this efficiently, compilers use a **two-buffer scheme** with **sentinels**.

3.2.1 Buffer Pairs

- Two buffers, each of size N (commonly a disk-block size, e.g. 4096 bytes), reloaded alternately.
- One read call loads N characters at once (cheaper than one system call per character).
- If fewer than N characters remain, the special marker eof fills the rest (it can never be a real source character).

Two pointers: - lexemeBegin — start of the current lexeme. - forward — scans ahead until a full match is found.

Once a lexeme is found: forward stops right after it; the lexeme is recorded; then lexemeBegin jumps to right after the lexeme, ready for the next round.

Problem: every time forward moves, two tests are needed — (1) end of buffer? (2) what character is this? This is solved using **sentinels**.

Simulation — lookahead for $\leq$ vs $<$

Input: a $\leq$ b

forward position	Character read	Decision
1	a	letter → part of an id; next char is space → id = a complete
2	(space)	skip
3	<	could start < or $\leq$; look ahead one more character
4	=	matches → token is $\leq$ (not <), consume both chars
5	(space)	skip
6	b	id = b

This shows why lookahead (and therefore buffering) is essential — a decision about a lexeme sometimes needs one extra character beyond it.

3.2.2 Sentinels

Add a **sentinel character** (eof) at the very end of *each* buffer. This merges the “end of buffer?” test and the “which character?” test into **one** check (a single switch on the character).

Simulation — sentinel-based reading

Buffer 1 ends with: ... C * * eof Buffer 2 starts with: 2 eof ...

Step	forward reads	Action
1	C	normal character
2	*	normal character
3	*	normal character
4	eof (at end of Buffer 1)	this eof is the sentinel → reload Buffer 2, move forward to its start
5	2	normal character
6	eof (appears inside Buffer 2, not at its edge)	this means the real end of input has been reached → terminate lexical analysis

Rule: an eof at the exact edge of a buffer = “reload the other buffer.” An eof appearing anywhere else = “true end of source file.”

Note — Implementing multiway branches

The switch on the character under forward looks like it could be slow (checking case after case), but the order in which cases are listed does not actually matter for speed — in practice, a multiway branch on a character is implemented in **one step**, by jumping to an address stored in an array of addresses indexed by the character itself (a jump table).

Note — Can we run out of buffer space?

In most languages, lexemes are short, so a buffer size N in the thousands is enough. Risk case: very long string literals spanning many lines could exceed N (in Java, this is why long strings are conventionally written as several pieces joined with +). A harder case: some languages (like PL/I) don’t reserve keywords — `DECLARE(ARG1, ARG2, ...)` could be a keyword statement or a call to a procedure named `DECLARE`, and arbitrarily long lookahead may be needed to decide. Modern languages avoid this by reserving their keywords.

Exam Q&A

Q22. Why are two buffers used instead of one? A: So one buffer can be scanned while the other is reloaded, allowing scanning to continue smoothly across buffer boundaries.

Q23. What do lexemeBegin and forward do? A: lexemeBegin marks the start of the current lexeme; forward scans ahead (with lookahead if needed) until the lexeme's true end is found.

Q24. What problem do sentinels solve? A: They remove the need for a separate buffer-end test on every character by combining it with the character test in a single switch statement.

Q25. When does an eof character mean the real end of input, versus just the end of one buffer? A: If eof appears exactly at a buffer's edge → reload the other buffer. If eof appears anywhere else (inside a buffer) → it marks genuine end of input.

3.3 Specification of Tokens

Regular expressions are the standard notation for specifying token patterns (lexeme forms). They cannot express every possible pattern, but are very effective for tokens like identifiers, numbers, and operators.

3.3.1 Strings and Languages

Term	Meaning
Alphabet	A finite set of symbols, e.g. $\{0, 1\}$, ASCII, Unicode.
String	A finite sequence of symbols from an alphabet. $ s $ = length of s .
Empty string (ϵ)	String of length 0.
Language	Any countable set of strings over an alphabet.

Parts of a string (example: banana)

Term	Meaning	Example
Prefix	remove zero or more symbols from the end	ban
Suffix	remove zero or more symbols from the beginning	nana
Substring	remove a prefix and a suffix (both ends)	nan
Subsequence	delete symbols from any positions	baan

Proper prefix/suffix/substring $\neq \epsilon$ and $\neq$ the string itself.

Concatenation: $xy = x$ followed by y . Example: dog + house = doghouse. $\epsilon s = s\epsilon = s$.

Exponentiation: $s^0 = \epsilon$, $s^i = s^{(i-1)}s \rightarrow$ so $s^1 = s$, $s^2 = ss$, etc.

3.3.2 Operations on Languages

Operation	Definition
Union $L \cup M$	$\{s \mid s \in L \text{ or } s \in M\}$
Concatenation LM	$\{st \mid s \in L, t \in M\}$
Kleene closure L^*	$L^0 \cup L^1 \cup L^2 \cup \dots$ (zero or more; $L^0 = \{\epsilon\}$)
Positive closure L^+	$L^1 \cup L^2 \cup \dots$ (one or more; no ϵ unless already in L)

	L^*	L^+
Occurrences	zero or more	one or more
Includes ϵ ?	always	only if $\epsilon \in L$

Example (L = letters, D = digits)

Expression	Meaning
$L \cup D$	all letters and digits
LD	one letter followed by one digit
L^4	all 4-letter strings
L^*	all strings of letters, including ϵ
$L(L \cup D)^*$	letter followed by any mix of letters/digits — shape of an identifier
D^+	one or more digits

Exam Q&A

Q26. Define union, concatenation, Kleene closure, and positive closure of languages. A: See table above.

Q27. Difference between Kleene closure and positive closure. A: L^* allows zero occurrences (includes ϵ); L^+ requires at least one occurrence (excludes ϵ unless it is already in L).

3.3.3 Regular Expressions

A **regular expression** describes a language using union, concatenation, and closure.

Basis rules: 1. ϵ is a regular expression for $\{\epsilon\}$. 2. Any symbol a in the alphabet is a regular expression for $\{a\}$.

Induction rules (r, s are regular expressions):

Form	Language
$r \mid s$	$L(r) \cup L(s)$

Form	Language
rs	$L(r)L(s)$
r^*	$(L(r))^*$
(r)	$L(r)$ (grouping only)

Precedence (highest → lowest): $*$ > concatenation > $|$ So $a|b^*c = a | (b^*c)$, not $(a|b)^*c$.

Examples ($\Sigma = \{a, b\}$)

Regex	Language
$a b$	$\{a, b\}$
$(a b)(a b)$	$\{aa, ab, ba, bb\}$
a^*	$\{\epsilon, a, aa, \dots\}$
$(a b)^*$	all strings of a's and b's, incl. ϵ
$a a^*b$	$\{a, b, ab, aab, \dots\}$

Algebraic laws

Law	Meaning
$r s = s r$	$ $ commutative
$r (s t) = (r s) t$	$ $ associative
$r(st) = (rs)t$	concatenation associative
$r(s t) = rs rt$	concatenation distributes over $ $
$\epsilon r = r\epsilon = r$	ϵ is identity for concatenation
$r^* = (r \epsilon)^*$	ϵ guaranteed in closure
$r^{**} = r^*$	$*$ is idempotent

Simulation — matching temp123 against `id = letter_(letter_|digit)*`

Step	Character	Matched by	Running match
1	t	letter_ (first part)	t
2	e	letter_ (inside *)	te
3	m	letter_	tem
4	p	letter_	temp
5	1	digit (inside *)	temp1
6	2	digit	temp12
7	3	digit	temp123
—	(end of lexeme)	—	full match: temp123 accepted as id

lsum is **rejected** as id because the first character 1 is a digit, not letter_.

Exam Q&A

Q28. Define regular expression. A: A notation, built from union, concatenation, and closure, used to describe the set of lexemes (pattern) matching a token.

Q29. State operator precedence for regular expressions. A: * (highest) $\rightarrow$ concatenation $\rightarrow$ | (lowest).

Q30. What is a regular set? When are two regular expressions equivalent? A: A language definable by a regular expression is a regular set. $r = s$ if both describe the same regular set (e.g. $a|b = b|a$).

Q31. Write the regular expression for a C identifier. A: `letter_ ( letter_ | digit )*` — must start with a letter/underscore; remaining characters can be letters, digits, or underscore.

Q32. State any three algebraic laws of regular expressions. A: $r|s = s|r$; $\epsilon r = r\epsilon = r$; $r^{**} = r^*$ (see full table above).

3.3.4 Regular Definitions

A **regular definition** names a regular expression so it can be reused in later expressions.

```
d1 -> r1
d2 -> r2
...
dn -> rn
```

Each r_i may use only the alphabet and **previously defined** names (no recursion allowed).

Regular definition — C identifier

```
letter_ -> A | B | ... | Z | a | b | ... | z | _
digit   -> 0 | 1 | ... | 9
id      -> letter_ ( letter_ | digit )*
```

Regular definition — unsigned number

```
digit      -> 0 | 1 | ... | 9
digits     -> digit digit*
optionalFraction -> . digits | ε
optionalExponent -> ( E ( + | - | ε ) digits ) | ε
number     -> digits optionalFraction optionalExponent
```

Simulation — matching 6.02E23 against number

Part of number	Consumes	Remaining input
digits	6	.02E23
optionalFraction (. digits)	.02	E23
optionalExponent (E (+ - ε) digits)	E23	(empty)
Result	full string matched	accepted

Simulation — why 1. is rejected

Part	Consumes	Remaining
digits	1	.
optionalFraction needs . digits (at least one digit after the dot)	fails — no digit after .	—
Result	—	rejected (1. is invalid; 1.0 would be valid)

Exam Q&A

Q33. What is a regular definition? Why is it used? A: A sequence of named regular expressions, used to shorten and simplify complex patterns by reusing names instead of repeating expressions.

Q34. Why can't regular definitions be recursive? A: Each r_i can use only previously defined names, preventing infinite recursion and ensuring every definition can be expanded into a plain regular expression.

Q35. Write the regular definition for unsigned numbers (e.g. 5280, 0.01234, 6.336E4). A: See box above.

Q36. Why does number not match 1. but does match 1.0? A: optionalFraction requires at least one digit after the decimal point; 1. has none, so it fails.

3.3.5 Extensions of Regular Expressions

Shorthand notations (introduced in tools like Lex) that make regex writing easier:

Extension	Meaning	Example
r^+	one or more repetitions (positive closure)	$a^+ \rightarrow a, aa, \dots$
$r^?$	zero or one occurrence ($r \mid \epsilon$)	$a^? \rightarrow \epsilon, a$
$[a_1a_2\dots a_n]$	character class = $a_1 \mid a_2 \mid \dots \mid a_n$	$[abc] = a \mid b \mid c$
$[a-z]$	range shorthand	$[a-z] = a \mid b \mid \dots \mid z$

Useful identities: $r^* = r^+ \mid \epsilon$ and $r^+ = r r^* = r^* r$.

Simplified regular definitions using shorthand

```

letter_ -> [A-Za-z_]
digit   -> [0-9]
id       -> letter_ ( letter_ | digit ) *

digits  -> digit+
number  -> digits ( . digits )? ( E [+ -]? digits )?

```

Note on Lex/Flex: keyword patterns must be listed **before** the identifier pattern, because the generator picks the *first* pattern in the list that matches — otherwise keywords like `if` would wrongly match the `id` pattern.

Exam Q&A

Q37. What does the + operator mean? How is it different from *? A: + = one or more occurrences (excludes ϵ unless already in the language); * = zero or more occurrences (always includes ϵ).

Q38. What does the ? operator mean? A: Zero or one occurrence; $r?$ = $r \mid \epsilon$.

Q39. What is a character class? Give an example. A: Shorthand for a union of single characters, e.g. $[abc] = a \mid b \mid c$, $[a-z] = a \mid b \mid \dots \mid z$.

3.3.6 Practice Exercises (Section 3.3)

- Determine input alphabet, numeric-constant form, and identifier form for: C, C++, C#, Fortran, Java, Lisp, SQL.
- Describe the languages of: $a(a \mid b)^*a$, $((\epsilon \mid a)b^*)^*$, $(a \mid b)^*a(a \mid b)(a \mid b)$, $a^*ba^*ba^*ba^*$, and (harder) $(aa \mid bb)^*((ab \mid ba)(aa \mid bb)^*(ab \mid ba)(aa \mid bb)^*)^*$.
- For a string of length n , count: prefixes, suffixes, proper prefixes, substrings, subsequences.
- Write a case-insensitive regular expression for SQL's SELECT.
- Write regular definitions for: strings containing the 5 vowels in order; strings with letters in ascending order; comments delimited by `/*` and `*/`; strings of digits with no repeated digit; strings of digits with at most one repeated digit; strings of a's/b's with an even number of a's and odd number of b's; informal chess-move notation (e.g. p-k4); strings of a's/b's that do **not** contain the substring `abb`; strings of a's/b's that do **not** contain the subsequence `abb`.
- Write character classes for: first 10 letters (a-j) in either case; lowercase consonants; hexadecimal digits; sentence-ending punctuation.
- (Lex notation) Practice: quoting operator characters, complemented classes $[\sim \dots]$, repetition $\{m, n\}$, anchors $\wedge, \$$, shell filename patterns, SQL LIKE patterns.

3.4 Recognition of Tokens (Introduction)

After specifying patterns with regular expressions, the next step is to build code (or an automaton) that scans the input and finds the next lexeme matching one of the patterns.

Example grammar (branching statements, Pascal-like):

```
stmt -> if expr then stmt
      | if expr then stmt else stmt
      | ε
expr  -> term relop term
      | term
term  -> id
      | number
```

Simulation — tokenizing if a < b then x = 1

Step	Lexeme found	Token
1	if	keyword IF
2	a	id
3	<	relop (LT)
4	b	id
5	then	keyword THEN
6	x	id
7	=	assign_op
8	1	number

Token stream: IF id(a) relop(<) id(b) THEN id(x) assign_op number(1)

This matches the `stmt -> if expr then stmt` rule, where `expr -> term relop term` (`a < b`) and the inner `stmt` is an assignment.

Exam Q&A

Q40. What is the next step after specifying token patterns using regular expressions? A: Build a program (based on automata — NFA then DFA) that scans the input and identifies, at each point, the next lexeme matching one of the patterns; this is the core of an actual lexical analyzer.

Q41. In the grammar `stmt -> if expr then stmt | if expr then stmt else stmt | ε`, what tokens would appear for an if statement? A: IF, tokens for `expr` (e.g. id, relop, number), THEN, tokens for the nested `stmt`, and optionally ELSE with another `stmt`.